Clasificación de Modelos de Autos con Deep Learning:

CNN propia contra Transfer Learning con EfficientNetB0

Alfredo Alejandro Soto Herrera – A01711368

Abstract – En el presente documento se implementan dos modelos de aprendizaje profundo con TensorFlow/Keras para clasificar fotografías de 10 modelos de autos deportivos (Audi R8, Audi RS5, BMW M3, BMW M5, Chevrolet Camaro, Ford Mustang, Mercedes-AMG GT, Nissan GT-R, Porsche 911 y Toyota Supra) a partir de un dataset real de Kaggle. Se realizó un EDA y un ETL específicos para imágenes (detección de duplicados e imágenes que no son autos, análisis del desbalance y división estratificada sin fuga de información), que redujeron el dataset de 2,084 a 1,807 imágenes. El primer modelo es una red convolucional diseñada desde cero, que alcanzó 31.5 % de accuracy en validación; tras probar ajustes (pesos por clase y mayor profundidad) ninguno superó el margen de error. El segundo modelo utiliza transfer learning con EfficientNetB0 preentrenada en ImageNet y fine-tuning en dos fases, con el que se obtuvo 62.8 % de accuracy y 0.633 de macro-F1 en validación.

Keywords – IA, Deep Learning, CNN, Transfer Learning, EfficientNet, TensorFlow, Clasificación de imágenes, EDA, ETL, Overfitting.

Documento hecho el 3 de noviembre del 2026.

I. INTRODUCCIÓN

La clasificación de imágenes es una de las tareas donde el aprendizaje profundo ha tenido más impacto. Un momento clave fue en 2012, cuando AlexNet, una red neuronal convolucional (CNN), ganó la competencia ImageNet con un margen muy amplio sobre los métodos tradicionales [1]. Desde entonces, las CNN se han convertido en la base de la visión por computadora moderna [2].

Reconocer el modelo de un auto (por ejemplo, distinguir un BMW M3 de un BMW M5) es un problema de clasificación de grano fino, las clases se parecen mucho entre sí y las diferencias están en detalles como la parrilla, los faros o las proporciones de la carrocería [3]. Esto lo hace más difícil que distinguir categorías generales como “auto” contra “avión”.

Además, entrenar una red profunda desde cero requiere muchas imágenes. Cuando se cuenta con pocos datos, una alternativa es el transfer learning: reutilizar una red ya entrenada en un dataset grande (como ImageNet [5]) y adaptarla al nuevo problema, ya que los primeros niveles de una CNN aprenden rasgos generales que son transferibles [4].

El objetivo de este trabajo es tomar un dataset real de Kaggle con fotografías de 10 modelos de autos [6], prepararlo con un EDA y un ETL, y comparar dos aproximaciones con TensorFlow [7]: (1) una CNN diseñada desde cero y (2) una red preentrenada con transfer learning. Se evalúa el desempeño de la aproximación inicial, se realizan ajustes para mejorarla y se documentan las decisiones tomadas.

II. EDA Y ETL

A. EDA inicial

El EDA (Análisis Exploratorio de los Datos) sirve para conocer con qué se está trabajando antes de transformarlo. En un dataset de imágenes se revisan aspectos distintos a los de un dataset tabular: cuántas fotos hay por clase, sus tamaños y formatos, si hay archivos dañados o repetidos y si todas muestran realmente lo que deben.

El dataset [6] contiene 2,084 imágenes JPEG en modo RGB, organizadas en 10 carpetas (una por modelo de auto) y un archivo metadata.csv con la fuente, el autor y la licencia de cada foto. Ninguna imagen estaba corrupta y todas coincidían con el archivo de metadatos. Casi todas provienen de Flickr, es decir, son fotografías reales con calidad, ángulos y condiciones de luz muy variables.

Las clases no están balanceadas: la mayor es BMW M5 con 239 imágenes y la menor es Audi RS5 con 124, una razón de desbalance de 1.93 (Figura 1). No es grave, pero implica usar un split estratificado y vigilar métricas por clase (F1) además del accuracy.

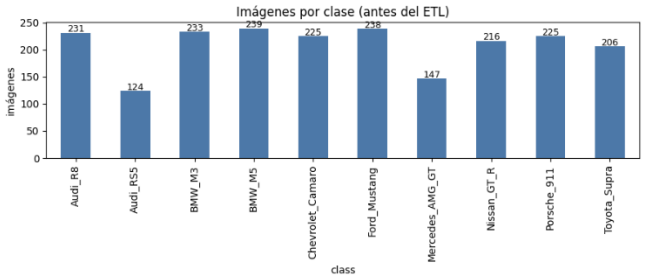


Figura 1. Distribución de imágenes por clase (antes del ETL).

Las imágenes no tienen el mismo tamaño ni la misma proporción: la mayoría es apaisada (1.5:1), pero hay verticales y panorámicas, con relaciones de aspecto de aproximadamente 0.56 hasta 3.4 (Figura 2). Como una CNN necesita entradas de tamaño fijo, habrá que reescalarlas sin deformar el auto.

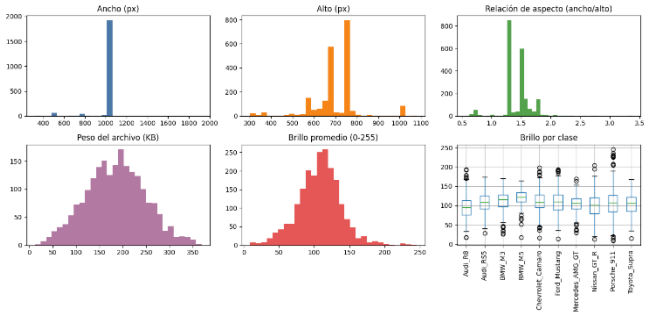


Figura 2. Distribución de tamaño, proporción, peso y brillo de las imágenes.

Se revisaron muestras de cada clase (Figura 3) y los casos extremos. Las imágenes más oscuras (autos de noche o fotos de estudio) y las proporciones extremas son válidas, por lo que no se eliminaron automáticamente. En cambio, al ordenar las imágenes por la fracción de píxeles casi blancos se encontraron contenidos que no son una fotografía de un auto: diagramas y bocetos técnicos de motores, modelos de LEGO o juguete, plantillas de decoración y anuncios de revista (Figura 4).



Figura 3. Muestras de cada clase del dataset.



Figura 4. Imágenes que no son fotografías reales de un auto.

Se calculó el MD5 de cada archivo (duplicados exactos) y un perceptual hash (pHash) [8], que detecta imágenes visualmente iguales, aunque estén recomprimidas o reescaladas. Se encontraron 9 duplicados exactos y 14 pares con distancia de pHash menor o igual a 6, de los cuales 6 pares estaban en clases distintas. Al revisarlos se observó que suelen ser fotos que muestran dos autos a la vez (por ejemplo, un M3 junto a un M5 o un Camaro junto a un Mustang), lo que genera etiquetas ambiguas (Figura 5).



Figura 5. Ejemplos de duplicados exactos, incluyendo fotos en clases distintas.

Finalmente, una de las imágenes provenía de Sketchfab, una plataforma de modelos 3D, por lo que no es una fotografía real y se descartó por el requisito de utilizar datos reales.

B. ETL

Con base en el EDA, el proceso de transformación consistió en los siguientes pasos (Tabla 1):

- Eliminar imágenes que no son fotografías reales.
- Eliminar duplicados**.** Si una misma imagen aparecía varias veces en la misma clase, se conservó una. Si aparecía en clases distintas, se eliminaron todas las copias porque su etiqueta es ambigua.
- Revisión visual de las imágenes que no son un auto real o completo (diagramas, modelos de juguete, anuncios, interiores y similares.
- Agrupar casi duplicados (pHash $\leq$ 6) para que nunca queden separados entre train, val y test.
- División estratificada por grupos en 70 % entrenamiento, 15 % validación y 15 % prueba, con semilla fija (42), usando StratifiedGroupKFold de 20 pliegues (14 para train, 3 para val y 3 para test). Estratificar mantiene la proporción de cada clase.
- Reescalar todas las imágenes a 224×224 píxeles en RGB, conservando la proporción del auto y rellenando con negro (padding), en lugar de deformarlo a un cuadrado.
- Calcular pesos por clase a partir del conjunto de entrenamiento, para compensar el desbalance.

La normalización de los píxeles y el data augmentation no se hicieron en el ETL, sino dentro de los modelos, porque el augmentation solo debe aplicarse durante el entrenamiento.

Paso del ETL	Imágenes
Archivos corruptos	0
No son fotos reales (fuente ≠ Flickr)	1
Duplicados en la misma clase (se conserva 1)	4
Duplicados entre clases distintas (se eliminan todas)	12
Revisión visual (diagramas, juguetes, anuncios, etc.)	260
Total eliminadas	277
Imágenes finales (de 2,084)	**1,807 (86.7 %)**

Tabla 1. Imágenes eliminadas en cada paso del ETL.

C. EDA final

El EDA final sirve para corroborar que el ETL salió como se esperaba. El dataset quedó con 1,807 imágenes: 1,267 para entrenamiento (70.1 %), 270 para validación (14.9 %) y 270 para prueba (14.9 %), con la distribución por clase de la Tabla 2 (Figura 6). Todas las imágenes procesadas son de 224×224 en modo RGB y ninguna presentó errores al abrirse.

Clase	Train	Val	Test	Total
Audi R8	153	31	30	214
Audi RS5	82	17	19	118
BMW M3	148	33	32	213
BMW M5	163	35	35	233
Chevrolet Camaro	126	25	25	176
Ford Mustang	118	27	24	169
Mercedes-AMG GT	94	21	24	139
Nissan GT-R	123	26	25	174
Porsche 911	124	25	26	175
Toyota Supra	136	30	30	196
Total	**1,267**	**270**	**270**	**1,807**

Tabla 2. Imágenes por clase y por conjunto después del ETL.

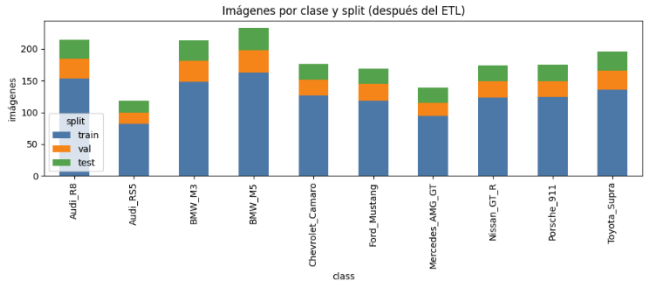


Figura 6. Distribución por clase y conjunto después del ETL.

También se verificó que no hubiera fuga de datos, para cada imagen de validación y de prueba se calculó la distancia pHash mínima contra las imágenes de entrenamiento, y en ambos casos fue de 10, es decir, ninguna imagen de val o test tiene un casi duplicado (distancia ≤ 6) en train. Por lo tanto, las métricas que se obtengan serán honestas. La Figura 7 muestra ejemplos de las imágenes ya procesadas.



Figura 7. Muestras de imágenes después del ETL.

III. MODELO 1: CNN

El primer modelo es una red convolucional diseñada desde cero. Su función es ser una línea base honesta contra la cual medir las mejoras posteriores, por lo que sus hiperparámetros iniciales no se ajustaron a propósito.

A. Pipeline de datos y data augmentation

Las imágenes se cargan desde las carpetas train/val/test con tf.data, en lotes de 32, con etiquetas enteras y pérdida de entropía cruzada categórica dispersa. Solo el conjunto de entrenamiento se baraja. Con 1,267 imágenes de entrenamiento el principal riesgo es el sobreajuste, por lo que se aplica data augmentation suave dentro del modelo (solo en entrenamiento): volteo horizontal, rotación (±5 %), zoom (10 %), contraste (10 %) y brillo (10 %). No se usa volteo vertical porque un auto de cabeza no aparece en la realidad.

B. Arquitectura

La red (Tabla 3) tiene cuatro bloques convolucionales con 32, 64, 128 y 256 filtros. Cada bloque aplica una convolución 3×3, normalización por lotes (BatchNorm) [10], activación ReLU y max pooling. En lugar de aplanar el mapa de características se usa global average pooling [15], que reduce de forma importante el número de parámetros y el riesgo de sobreajuste, seguido de dropout [11] con tasa 0.4 y una capa densa de 10 neuronas con softmax. En total son 392,426 parámetros.

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
augmentation (Sequential)	(None, 224, 224, 3)	0
rescaling (Rescaling)	(None, 224, 224, 3)	0
conv2d (Conv2D)	(None, 224, 224, 32)	864
batch_normalization (BatchNormalization)	(None, 224, 224, 32)	128
re_lu (ReLU)	(None, 224, 224, 32)	0
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 64)	18,432
batch_normalization_1 (BatchNormalization)	(None, 112, 112, 64)	256
re_lu_1 (ReLU)	(None, 112, 112, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_2 (Conv2D)	(None, 56, 56, 128)	73,728
batch_normalization_2 (BatchNormalization)	(None, 56, 56, 128)	512
re_lu_2 (ReLU)	(None, 56, 56, 128)	0
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)	0
conv2d_3 (Conv2D)	(None, 28, 28, 256)	294,912
batch_normalization_3 (BatchNormalization)	(None, 28, 28, 256)	1,024
re_lu_3 (ReLU)	(None, 28, 28, 256)	0
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 256)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 256)	0
dropout (Dropout)	(None, 256)	0
dense (Dense)	(None, 10)	2,570

Figura 8. Arquitectura de la CNN propia.

Capa	Configuración
Entrada + augmentation	224×224×3; volteo, rotación, zoom, contraste, brillo
Rescaling	Píxeles a rango 0-1
Bloque × 4	Conv 3×3 (32, 64, 128, 256) → BatchNorm → ReLU → MaxPool
GlobalAveragePooling2D	Un valor por mapa de características
Dropout	Tasa 0.4
Dense (softmax)	10 clases

Tabla 3. Arquitectura de la CNN propia.

C. Parámetros e hiperparámetros

Se utilizó el optimizador Adam [12] con learning rate de 1e-3, un valor estándar como punto de partida, y la pérdida sparse categorical crossentropy. Se entrenó hasta un máximo de 60 épocas con EarlyStopping (paciencia de 12 épocas sobre la pérdida de validación, restaurando los mejores pesos) y ReduceLROnPlateau (reduce el learning rate a la mitad si la validación se estanca durante 5 épocas).

D. Resultados

La aproximación inicial se detuvo por early stopping en la época 49, con la mejor época en la 37. En validación obtuvo 30.4 % de accuracy y 0.238 de macro-F1, mientras que en entrenamiento (medido sin augmentation) llegó a 41 % (Figura 9). Como referencia, adivinar al azar con 10 clases daría 10 %, así que el modelo sí aprendió, pero con un desempeño bajo.

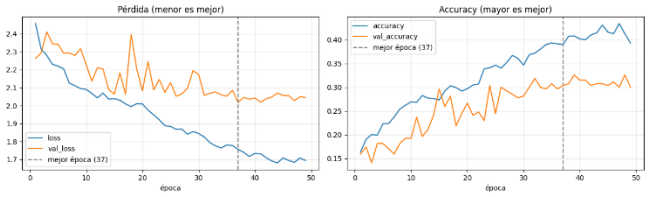


Figura 9. Curvas de aprendizaje de la CNN base.

La matriz de confusión (Figura 10) muestra que el modelo tiende a predecir BMW M5 como respuesta por defecto, esa clase tiene un recall de 0.86 pero una precisión de apenas 0.40, y 18 de los 33 BMW M3 de validación se clasificaron como M5. En cambio, las clases con menos imágenes de entrenamiento se detectan muy poco, Audi RS5 tuvo un recall de 0.06 y Mercedes-AMG GT de 0.10.

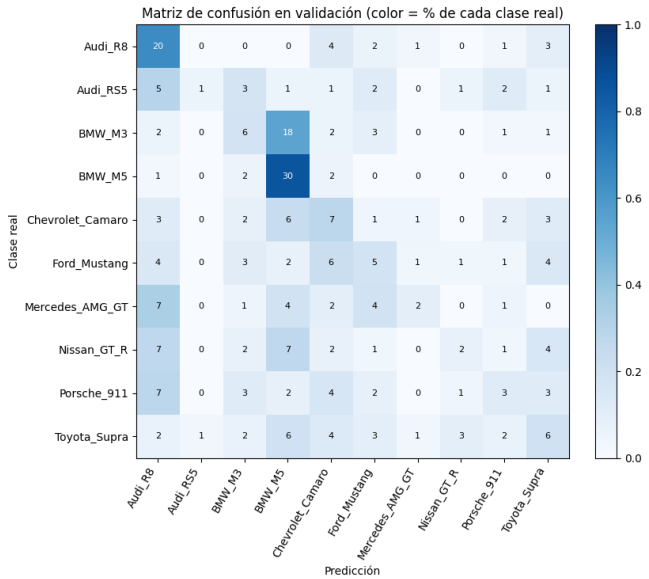


Figura 10. Matriz de confusión de la CNN base en validación.

Un accuracy de entrenamiento de solo 41 % indica un sesgo alto, la red es demasiado limitada para el problema. Además, desde aproximadamente la época 30 la pérdida de entrenamiento siguió bajando mientras la de validación se mantuvo plana, lo que apunta a un inicio de sobreajuste.

IV. PRUEBAS Y AJUSTES AL MODELO 1

Con el diagnóstico anterior se propuso una ronda de ajustes, cambiando una cosa a la vez para poder atribuir cada efecto a su causa:

- E0 (base): La CNN base reentrenada en las mismas condiciones para una comparación justa.
- E1 (pesos por clase): Ataca el desbalance, ponderando más los errores en las clases pequeñas (pesos de 0.78 para BMW M5 hasta 1.55 para Audi RS5).
- E2 (más profunda): 2 convoluciones por bloque en lugar de una (1,177,706 parámetros), para atacar el sesgo alto.
- E3 (combinada): E1 y E2 juntas.

Como el conjunto de validación tiene solo 270 imágenes, el accuracy tiene un margen de error de aproximadamente ±5.5 puntos (intervalo de confianza del 95 % de una proporción), por lo que diferencias menores pueden ser azar. Por eso cada experimento se repitió con dos semillas (42 y 123) y se reporta el promedio y la desviación estándar (Tabla 4).

Exp.	Train acc (%)	Val acc (%)	Val macro-F1 (%)	Brecha (pts)
E0 base	40.1 ± 0.6	31.5 ± 0.5	27.3 ± 0.3	8.7
E1 pesos	45.8 ± 3.9	32.0 ± 1.8	27.9 ± 1.9	13.7
E2 profunda	49.6 ± 1.5	30.4 ± 3.1	28.7 ± 2.5	19.3
E3 combinada	41.2 ± 7.8	30.0 ± 2.6	26.5 ± 5.2	11.2

Tabla 4. Resultados de los ajustes al modelo 1 (promedio ± desviación estándar de 2 semillas).

Los resultados muestran que ningún ajuste superó a la línea base de forma demostrable, todos quedaron alrededor de 30-32 % de accuracy, con diferencias de uno o dos puntos, muy por debajo del margen de error. Además:

- Más capacidad (E2) no ayudó, el accuracy de entrenamiento subió a casi 50 %, pero la validación no se movió y la brecha entre ambos pasó de 9 a 19 puntos, es decir, la red solo memoriza más.
- Los pesos por clase (E1) no resolvieron el desbalance de forma clara: Audi RS5 sigue siendo de las clases con peor F1.
- La combinación (E3) fue la más inestable (desviación estándar de 5.2 puntos en macro-F1), en una de las semillas el early stopping detuvo el entrenamiento hacia la época 30 por un pico en la pérdida de validación.

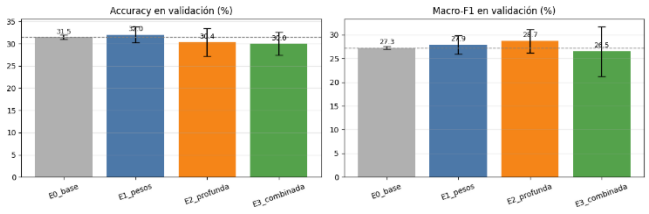


Figura 11. Comparación de los ajustes al modelo 1.

Estos resultados sugieren que el cuello de botella no son los hiperparámetros, sino que con alrededor de 1,300 fotos de entrenamiento una CNN entrenada desde cero no logra aprender los rasgos finos que distinguen a cada modelo de auto. Por esta razón no se siguió ajustando este modelo y se pasó a transfer learning. Para las comparaciones posteriores, el modelo 1 de referencia es **E0 (base)**, por ser el más simple sin que ningún ajuste lo supere de forma demostrable.

	E0_base	E1_pesos	E2_profunda	E3_combinada
Audi_R8	0.38	0.42	0.42	0.42
Audi_RS5	0.11	0.22	0.23	0.27
BMW_M3	0.24	0.25	0.19	0.09
BMW_M5	0.57	0.55	0.46	0.57
Chevrolet_Camaro	0.31	0.28	0.32	0.19
Ford_Mustang	0.25	0.21	0.30	0.20
Mercedes_AMG_GT	0.28	0.24	0.34	0.38
Nissan_GT_R	0.14	0.13	0.11	0.17
Porsche_911	0.16	0.23	0.20	0.17
Toyota_Supra	0.29	0.27	0.29	0.21
MACRO-F1	0.27	0.28	0.29	0.27

Figura 12. F1 por clase de los experimentos E0 a E3 en validación.

V. MODELO 2: TRANSFER LEARNING

A. Fundamento

El transfer learning consiste en aprovechar una red ya entrenada en una tarea grande y adaptarla a otra [4]. Se eligió EfficientNetB0 [9], una familia de CNN que escala conjuntamente profundidad, ancho y resolución, y que ofrece muy buen desempeño con relativamente pocos parámetros, preentrenada en ImageNet [5]. Esto es útil aquí porque la red ya sabe detectar bordes, texturas, formas y partes de objetos, y solo hay que enseñarle a combinarlos para reconocer 10 modelos de autos.

B. Arquitectura

El modelo se compone de: entrada → data augmentation → EfficientNetB0 sin su clasificador original → global average pooling → dropout (0.3) → capa densa de 10 neuronas con softmax. En total son 4,062,381 parámetros. Las imágenes se mantienen en el rango 0-255 ya que EfficientNet incluye su propio preprocesamiento. La base se invoca con training=False para que las capas BatchNorm se mantengan en modo inferencia y no se desestabilicen con un dataset pequeño.

C. Entrenamiento en dos fases

- Fase 1 (extracción de características**):** La base está congelada y solo se entrena la cabeza nueva. Adam con learning rate 1e-3, máximo 20 épocas y EarlyStopping con paciencia 6. Se usa un learning rate alto porque la cabeza empieza con pesos aleatorios.
- Fase 2 (fine-tuning**):** Se descongelan las últimas 40 capas de la base (excepto las BatchNorm), lo que da 2,050,330 parámetros entrenables, y se recompila con Adam y learning rate de 1e-5. Máximo 30 épocas, EarlyStopping con paciencia 8 y ReduceLROnPlateau. Un learning rate bajo evita destruir los rasgos aprendidos en ImageNet.

Los resultados se repitieron con las mismas dos semillas (42 y 123) y el mismo split que el modelo 1.

D. Resultados

El modelo 2 alcanzó en promedio **62.8 % ± 0.8** de accuracy y **0.633 ± 0.010** de macro-F1 en validación (Tabla 5), más del doble que el modelo 1. La fase 1 por sí sola llegó a 53.4 %, de modo que el fine-tuning aportó +9.4 puntos de accuracy y +10.2 de macro-F1.

Semilla	Fase 1 val acc	Final val acc	Final macro-F1	Train acc
42	53.0	63.3	64.0	82.7
123	53.7	62.2	62.6	83.2
Promedio	**53.4**	**62.8 ± 0.8**	**63.3 ± 1.0**	**83.0 ± 0.3**

Tabla 5. Resultados del modelo 2 por semilla (valores en %).

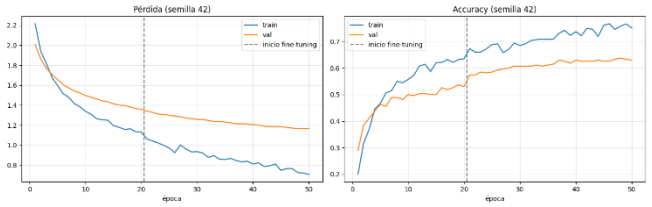


Figura 14. Curvas de entrenamiento del modelo 2 (fase 1 y fase 2).

En ambas fases el entrenamiento alcanzó el máximo de épocas permitido (20 y 30) y la pérdida de validación seguía bajando, es decir, el modelo no había convergido por completo. Además, hay una brecha de unos 20 puntos entre el accuracy de entrenamiento (83 %) y el de validación (63 %), señal de sobreajuste. Ambas observaciones motivan las mejoras de la sección siguiente.

En validación (semilla 42) el modelo cometió 99 errores de 270 imágenes. Las confusiones más frecuentes son entre modelos de la misma marca o muy parecidos (Figura 15): BMW M3 predicho como M5 (8 veces) y M5 como M3 (7), Ford Mustang como Chevrolet Camaro (4), Audi R8 como Toyota Supra (4), Porsche 911 como Toyota Supra (4),

Mercedes-AMG GT como Nissan GT-R (3) y Nissan GT-R como Toyota Supra (3).

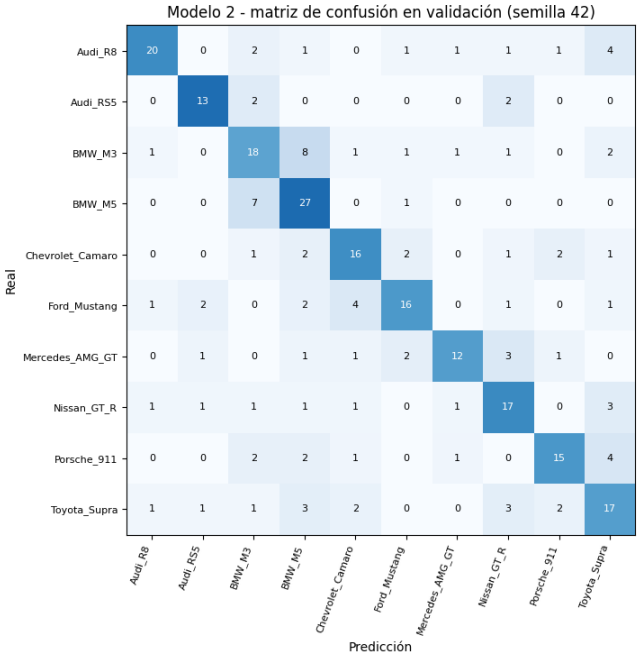


Figura 15. Matriz de confusión del modelo 2 en validación.

La comparación con el modelo 1 (Tabla 6, Figura 16) muestra una mejora de +31.3 puntos de accuracy y +36.0 de macro-F1, muy superior al margen de error de aproximadamente ±5.8 puntos de la validación, por lo que la mejora es real y no producto del azar.

Modelo	Parámetros	Train acc	Val acc	Val macro-F1
M1: CNN propia (E0)	392,426	40.1 ± 0.6	31.5 ± 0.5	27.3 ± 0.3
M2: EfficientNetB0	4,062,381	83.0 ± 0.3	62.8 ± 0.8	63.3 ± 1.0

Tabla 6. Comparación entre el modelo 1 y el modelo 2 (valores en %).

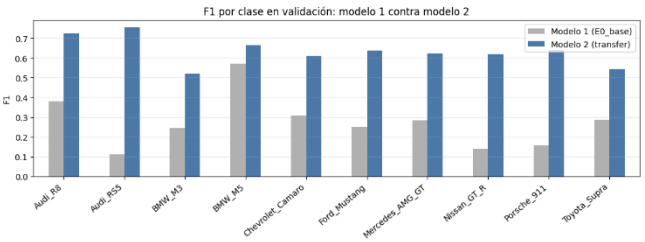


Figura 16. F1 por clase en validación: modelo 1 contra modelo 2.

E. Mejoras al modelo 2

A partir del diagnóstico anterior se propusieron los experimentos de la Tabla 7, cada uno justificado por una observación: entrenar más tiempo (el modelo no había convergido), descongelar más capas (el fine-tuning ayudó +9.4 puntos), regularizar con dropout 0.5, label smoothing [13] y mayor augmentation (sobreajuste), y probar una base más potente, ConvNeXtTiny [14] (confusiones entre modelos casi idénticos). También se evalúan dos mejoras sin costo de entrenamiento: test-time augmentation (promediar la predicción de la foto y de su versión volteada) y ensamble de semillas.

Exp.	Cambio	Val acc	Macro-F1
A	Entrenamiento más largo		
B	A + 100 capas descongeladas		
C	A + regularización		
D	B + C		
E	A con ConvNeXtTiny		

Tabla 7. Experimentos de mejora al modelo 2.

Figura 17. Resultados de las mejoras al modelo 2.

F. Evaluación final en test

El conjunto de prueba no se utilizó para ninguna decisión; se evalúa una sola vez con los modelos finales (Tabla 8).

Modelo	Test acc	Test macro-F1
Modelo 1: CNN propia (E0)		
Modelo 2: EfficientNetB0		
Modelo 2 mejorado		

Tabla 8. Resultados finales en el conjunto de prueba.

VI. DISCUSIÓN

La comparación muestra que, con un dataset pequeño y un problema de grano fino, el transfer learning supera con claridad a una CNN entrenada desde cero, 62.8 % contra 31.5 % de accuracy. Los ajustes al modelo 1 confirmaron que su techo no depende de los hiperparámetros, sino de la cantidad de datos y de la capacidad de aprender rasgos útiles desde cero. En cambio, la base preentrenada ya contaba con esos rasgos.

Aun así, el modelo 2 no es perfecto, la brecha de 20 puntos entre entrenamiento y validación indica sobreajuste, y las confusiones se concentran en modelos casi idénticos de una misma marca (M3 con M5, Camaro con Mustang). Parte de esa dificultad es inherente al problema y parte proviene de los datos, algunas fotos muestran dos autos y su etiqueta es ambigua.

VII. CONCLUSIÓN

Se preparó un dataset real de 10 modelos de autos mediante un EDA y un ETL específicos para imágenes, y se compararon dos aproximaciones de aprendizaje profundo con TensorFlow. La CNN (modelo 1) alcanzó 31.5 % de accuracy

y ninguno de los ajustes probados superó el margen de error. El transfer learning con EfficientNetB0 (modelo 2) alcanzó 62.8 % de accuracy y 0.633 de macro-F1, una mejora de más de 31 puntos que sí es significativa.

Un aprendizaje central del proyecto es que la calidad de los datos importa tanto como el modelo, el EDA permitió detectar duplicados con etiquetas contradictorias, imágenes que no eran autos y desbalance de clases, y evitar fuga de información entre los conjuntos, lo que hace confiables las métricas reportadas.

APÉNDICE

Repositorio del proyecto: AlejandroSH1/Implementacion-de-un-Modelo-de-DeepLearning

REFERENCIAS

[1] A. Krizhevsky, I. Sutskever y G. E. Hinton, "ImageNet classification with deep convolutional neural networks," en Advances in Neural Information Processing Systems 25 (NIPS 2012), 2012, pp. 1097–1105.

[2] Y. LeCun, Y. Bengio y G. Hinton, "Deep learning," Nature, vol. 521, pp. 436–444, 2015.

[3] J. Krause, M. Stark, J. Deng y L. Fei-Fei, "3D object representations for fine-grained categorization," en 4th IEEE Workshop on 3D Representation and Recognition (3dRR-13), ICCV Workshops, 2013.

[4] J. Yosinski, J. Clune, Y. Bengio y H. Lipson, "How transferable are features in deep neural networks?," en Advances in Neural Information Processing Systems 27 (NIPS 2014), 2014, pp. 3320–3328.

[5] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li y L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," en IEEE CVPR, 2009, pp. 248–255.

[6] lightangle, "Car Model Image Classification," Kaggle, dataset. [En línea]. Disponible: https://www.kaggle.com/datasets/lightangle/car-model-image-classification

[7] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous distributed systems," arXiv:1603.04467, 2016. Ver también F. Chollet et al., "Keras," 2015. [En línea]. Disponible: https://keras.io

[8] C. Zauner, "Implementation and benchmarking of perceptual image hash functions," tesis de maestría, Upper Austria University of Applied Sciences, Hagenberg, 2010.

[9] M. Tan y Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," en Proc. ICML, PMLR vol. 97, 2019, pp. 6105–6114.

[10] S. Ioffe y C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," en Proc. ICML, 2015, pp. 448–456.

[11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever y R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," J. Mach. Learn. Res., vol. 15, pp. 1929–1958, 2014.

[12] D. P. Kingma y J. Ba, "Adam: A method for stochastic optimization," en ICLR, 2015 (arXiv:1412.6980).

[13] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens y Z. Wojna, "Rethinking the Inception architecture for computer vision," en IEEE CVPR, 2016, pp. 2818–2826.

[14] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell y S. Xie, "A ConvNet for the 2020s," en IEEE/CVF CVPR, 2022, pp. 11976–11986.

[15] M. Lin, Q. Chen y S. Yan, "Network in network," en ICLR, 2014 (arXiv:1312.4400).